

Plasticity

Ralf Hielscher

Faculty of Mathematics and Computer Science,
TU Bergakademie Freiberg, Germany

MTEX Workshop 2024

The Velocity Gradient Tensor

describes the local rate of deformation

```
L = velocityGradientTensor.uniaxial(vector3d.Z)
```

```
L = velocityGradientTensor (xyz)  
rank: 2 (3 x 3)  
  
-0.5    0    0  
  0 -0.5    0  
  0    0    1
```

```
d_exp = vector3d.Y; d_compr = vector3d.X;  
L = velocityGradientTensor.pureShear(d_exp, d_compr)
```

```
d = vector3d.Y; n = vector3d.X;  
L = velocityGradientTensor.simpleShear(d, n)
```

```
rot = rotation.byAxisAngle(vector3d.Z, 10*degree)  
L = velocityGradientTensor.spin(rot)
```

The Velocity Gradient Tensor

describes the local rate of deformation

```
L = velocityGradientTensor.uniaxial(vector3d.Z)
```

```
d_exp = vector3d.Y; d_compr = vector3d.X;  
L = velocityGradientTensor.pureShear(d_exp, d_compr)
```

```
L = velocityGradientTensor (xyz)  
rank: 2 (3 x 3)
```

```
-2  0  0  
 0  2  0  
 0  0  0
```

```
d = vector3d.Y; n = vector3d.X;  
L = velocityGradientTensor.simpleShear(d, n)
```

```
rot = rotation.byAxisAngle(vector3d.Z, 10*degree)  
L = velocityGradientTensor.spin(rot)
```

The Velocity Gradient Tensor

describes the local rate of deformation

```
L = velocityGradientTensor.uniaxial(vector3d.Z)
```

```
d_exp = vector3d.Y; d_compr = vector3d.X;  
L = velocityGradientTensor.pureShear(d_exp, d_compr)
```

```
d = vector3d.Y; n = vector3d.X;  
L = velocityGradientTensor.simpleShear(d, n)
```

```
L = velocityGradientTensor (xyz)  
rank: 2 (3 x 3)  
  
0 0 0  
2 0 0  
0 0 0
```

```
rot = rotation.byAxisAngle(vector3d.Z, 10*degree)  
L = velocityGradientTensor.spin(rot)
```

The Velocity Gradient Tensor

describes the local rate of deformation

```
L = velocityGradientTensor.uniaxial(vector3d.Z)
```

```
d_exp = vector3d.Y; d_compr = vector3d.X;  
L = velocityGradientTensor.pureShear(d_exp, d_compr)
```

```
d = vector3d.Y; n = vector3d.X;  
L = velocityGradientTensor.simpleShear(d, n)
```

```
rot = rotation.byAxisAngle(vector3d.Z, 10*degree)  
L = velocityGradientTensor.spin(rot)
```

```
L = spinTensor (xyz)  
rank: 2 (3 x 3)
```

```
*10^-2  
      0  -17.453      0  
17.453      0      0  
      0      0      0
```

Decomposition of the Velocity Gradient Tensor

The deformation gradient tensor can be decomposed into a sum of a symmetric and an antisymmetric tensor

$$W = L \cdot \mathbf{antiSym}$$

$$D = L \cdot \mathbf{sym}$$

```
W = spinTensor (xyz)
```

```
rank: 2 (3 x 3)
```

```
0  -1  0
```

```
1   0  0
```

```
0   0  0
```

```
D = strainRateTensor (xyz)
```

```
rank: 2 (3 x 3)
```

```
0  1  0
```

```
1  0  0
```

```
0  0  0
```

Decomposition of the Velocity Gradient Tensor

The deformation gradient tensor can be decomposed into a sum of a symmetric and an antisymmetric tensor

$$W = L \cdot \mathbf{antiSym}$$

$$D = L \cdot \mathbf{sym}$$

the antisymmetric portion models the spin

rotation (W)

```
ans = rotation
```

```
size: 1 x 1
```

```
Bunge Euler angles in degree
```

```
phi1    Phi    phi2    Inv.
```

```
57.3     0       0       0
```

The Deformation Gradient Tensor $\mathbf{F}$

describes the local deformation in continuum mechanics and satisfies

$$\dot{\mathbf{F}}(t) = \mathbf{L}(t) \mathbf{F}(t)$$

In case of constant deformation $\mathbf{L}(t) = \mathbf{L}$ we obtain

$$\mathbf{F}(t) = \exp(t \cdot \mathbf{L})$$

$$\mathbf{F} = \text{expm}(\mathbf{L} * t)$$

In the case of time dependent deformation we have

$$\mathbf{F}(t) = \exp\left(\int_0^t \mathbf{L}(\tau) d\tau\right)$$

```
tau = linspace(0,t); d_tau = tau(2)-tau(1);  
L_tau = (t-tau) .* L  
F = expm(cumsum(L_tau * d_tau))
```


The Deformation Gradient Tensor $\mathbf{F}$

describes the local deformation in continuum mechanics and satisfies

$$\dot{\mathbf{F}}(t) = \mathbf{L}(t) \mathbf{F}(t)$$

In case of constant deformation $\mathbf{L}(t) = \mathbf{L}$ we obtain

$$\mathbf{F}(t) = \exp(t \cdot \mathbf{L})$$

$$\mathbf{F} = \text{expm}(\mathbf{L} * t)$$

In the case of time dependent deformation we have

$$\mathbf{F}(t) = \exp\left(\int_0^t \mathbf{L}(\tau) d\tau\right)$$

```
tau = linspace(0,t); d_tau = tau(2)-tau(1);  
L_tau = (t-tau) .* L  
F = expm(cumsum(L_tau * d_tau))
```

The Deformation Gradient Tensor $\mathbf{F}$

describes the local deformation in continuum mechanics and satisfies

$$\dot{\mathbf{F}}(t) = \mathbf{L}(t) \mathbf{F}(t)$$

In case of constant deformation $\mathbf{L}(t) = \mathbf{L}$ we obtain

$$\mathbf{F}(t) = \exp(t \cdot \mathbf{L})$$

$$\mathbf{F} = \text{expm}(\mathbf{L} * t)$$

```
F = deformationGradientTensor (xyz)
rank: 2 (3 x 3)

1 0 0
2 1 0
0 0 1
```

In the case of time dependent deformation we have

$$\mathbf{F}(t) = \exp\left(\int_0^t \mathbf{L}(\tau) d\tau\right)$$

```
tau = linspace(0, t); d_tau = tau(2) - tau(1);
```

The Deformation Gradient Tensor $\mathbf{F}$

describes the local deformation in continuum mechanics and satisfies

$$\dot{\mathbf{F}}(t) = \mathbf{L}(t) \mathbf{F}(t)$$

In case of constant deformation $\mathbf{L}(t) = \mathbf{L}$ we obtain

$$\mathbf{F}(t) = \exp(t \cdot \mathbf{L})$$

$$\mathbf{F} = \mathbf{expm}(\mathbf{L} * t)$$

In the case of time dependent deformation we have

$$\mathbf{F}(t) = \exp\left(\int_0^t \mathbf{L}(\tau) d\tau\right)$$

```
tau = linspace(0,t); d_tau = tau(2)-tau(1);  
L_tau = (t-tau) .* L  
F = expm(cumsum(L_tau * d_tau))
```

Decomposition of the Deformation Gradient Tensor

The deformation gradient tensor $\mathbf{F}$ can be decomposed into a rotational part $\mathbf{R}$, a right stretch tensor $\mathbf{U}$ and a left stretch tensor $\mathbf{V}$ such that

$$\mathbf{F} = \mathbf{R}\mathbf{U} = \mathbf{V}\mathbf{R}$$

$$[\mathbf{R}, \mathbf{V}, \mathbf{U}] = \text{polar}(\mathbf{F})$$

```
R = tensor (xyz)
rank: 2 (3 x 3)

0.7071  -0.7071    0
0.7071   0.7071    0
0         0        1
```

```
U = tensor (xyz)
rank: 2 (3 x 3)

2.1213  0.7071    0
0.7071  0.7071    0
0         0        1
```

The polar decomposition is closely related to the singular value decomposition of $\mathbf{F}$

Decomposition of the Deformation Gradient Tensor

The deformation gradient tensor $\mathbf{F}$ can be decomposed into a rotational part $\mathbf{R}$, a right stretch tensor $\mathbf{U}$ and a left stretch tensor $\mathbf{V}$ such that

$$\mathbf{F} = \mathbf{R}\mathbf{U} = \mathbf{V}\mathbf{R}$$

```
[R,V,U] = polar(F)
```

The polar decomposition is closely related to the singular value decomposition of $\mathbf{F}$

```
[v,s,u] = svd(expm(L .* tau))
```

```
a = s(1); b = s(3);  
plotEllipse([0,0],a,b,u(1).rho)
```

```
plotEllipse([0,0],a,b,v(1).rho)
```

Decomposition of the Deformation Gradient Tensor

The deformation gradient tensor $\mathbf{F}$ can be decomposed into a rotational part $\mathbf{R}$, a right stretch tensor $\mathbf{U}$ and a left stretch tensor $\mathbf{V}$ such that

$$\mathbf{F} = \mathbf{R}\mathbf{U} = \mathbf{V}\mathbf{R}$$

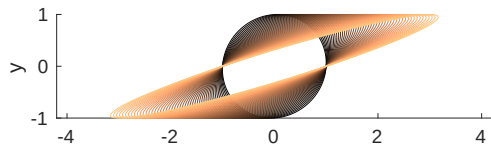
```
[R,V,U] = polar(F)
```

The polar decomposition is closely related to the singular value decomposition of $\mathbf{F}$

```
[v,s,u] = svd(expm(L .* tau))
```

```
a = s(1); b = s(3);  
plotEllipse([0,0],a,b,u(1).rho)
```

```
plotEllipse([0,0],a,b,v(1).rho)
```



Decomposition of the Deformation Gradient Tensor

The deformation gradient tensor $\mathbf{F}$ can be decomposed into a rotational part $\mathbf{R}$, a right stretch tensor $\mathbf{U}$ and a left stretch tensor $\mathbf{V}$ such that

$$\mathbf{F} = \mathbf{R}\mathbf{U} = \mathbf{V}\mathbf{R}$$

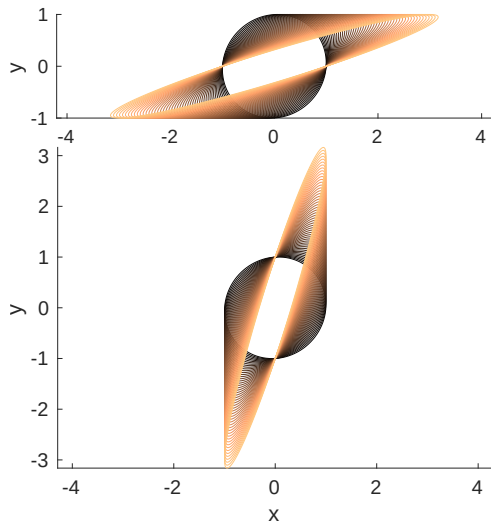
```
[R,V,U] = polar(F)
```

The polar decomposition is closely related to the singular value decomposition of $\mathbf{F}$

```
[v,s,u] = svd(expm(L .* tau))
```

```
a = s(1); b = s(3);  
plotEllipse([0,0],a,b,u(1).rho)
```

```
plotEllipse([0,0],a,b,v(1).rho)
```



Strain Tensors

right Cauchy-Green stretch tensor $\mathbf{C} = \mathbf{U}^2 = \mathbf{F}'\mathbf{F}$

$$\mathbf{C} = \mathbf{F}' \cdot * \mathbf{F}$$

```
C = tensor (xyz)
rank: 2 (3 x 3)
```

```
      1  0.0524      0
0.0524  1.0027      0
      0      0      1
```


Strain Tensors

right Cauchy-Green stretch tensor $\mathbf{C} = \mathbf{U}^2 = \mathbf{F}'\mathbf{F}$

$$\mathbf{C} = \mathbf{F}' \cdot * \mathbf{F}$$

left Cauchy-Green stretch tensor $\mathbf{B} = \mathbf{V}^2 = \mathbf{F}\mathbf{F}'$

$$\mathbf{B} = \mathbf{F} \cdot * \mathbf{F}'$$

```
B = tensor (xyz)  
rank: 2 (3 x 3)
```

```
1.0027  0.0524      0  
0.0524      1      0  
      0      0      1
```

Strain Tensors

right Cauchy-Green stretch tensor $\mathbf{C} = \mathbf{U}^2 = \mathbf{F}'\mathbf{F}$

$$\mathbf{C} = \mathbf{F}' \cdot * \mathbf{F}$$

left Cauchy-Green stretch tensor $\mathbf{B} = \mathbf{V}^2 = \mathbf{F}\mathbf{F}'$

$$\mathbf{B} = \mathbf{F} \cdot * \mathbf{F}'$$

Green-Lagrange strain tensor

$$\mathbf{E} = (\mathbf{C} - \mathbf{tensor.eye}) ./ 2$$

```
E = tensor (xyz)  
rank: 2 (3 x 3)
```

```
*10^-3
```

```
      0 26.204      0  
26.204  1.373      0  
      0      0      0
```

Strain Tensors

right Cauchy-Green stretch tensor $\mathbf{C} = \mathbf{U}^2 = \mathbf{F}'\mathbf{F}$

$$\mathbf{C} = \mathbf{F}' \cdot * \mathbf{F}$$

left Cauchy-Green stretch tensor $\mathbf{B} = \mathbf{V}^2 = \mathbf{F}\mathbf{F}'$

$$\mathbf{B} = \mathbf{F} \cdot * \mathbf{F}'$$

Green-Lagrange strain tensor

$$\mathbf{E} = (\mathbf{C} - \mathbf{tensor.eye}) ./ 2$$

Almansi strain tensor

$$\mathbf{e} = (\mathbf{tensor.eye} - \mathbf{inv}(\mathbf{B})) ./ 2$$

```
e = tensor (xyz)
rank: 2 (3 x 3)

*10^-3
      0 26.204      0
26.204 -1.373      0
      0      0      0
```

Strain Tensors

right Cauchy-Green stretch tensor $\mathbf{C} = \mathbf{U}^2 = \mathbf{F}'\mathbf{F}$

$$\mathbf{C} = \mathbf{F}' \cdot * \mathbf{F}$$

left Cauchy-Green stretch tensor $\mathbf{B} = \mathbf{V}^2 = \mathbf{F}\mathbf{F}'$

$$\mathbf{B} = \mathbf{F} \cdot * \mathbf{F}'$$

Green-Lagrange strain tensor

$$\mathbf{E} = (\mathbf{C} - \mathbf{tensor.eye}) ./ 2$$

Almansi strain tensor

$$\mathbf{e} = (\mathbf{tensor.eye} - \mathbf{inv}(\mathbf{B})) ./ 2$$

engineering strain

$$\mathbf{eps} = \mathbf{F.sym} - \mathbf{tensor.eye}$$

```
eps = tensor (xyz)
```

```
rank: 2 (3 x 3)
```

```
*10^-3
```

```
0 26.204 0
```

```
26.204 0 0
```

```
0 0 0
```

Strain Tensors

right Cauchy-Green stretch tensor $\mathbf{C} = \mathbf{U}^2 = \mathbf{F}'\mathbf{F}$

$$\mathbf{C} = \mathbf{F}' \cdot * \mathbf{F}$$

left Cauchy-Green stretch tensor $\mathbf{B} = \mathbf{V}^2 = \mathbf{F}\mathbf{F}'$

$$\mathbf{B} = \mathbf{F} \cdot * \mathbf{F}'$$

Green-Lagrange strain tensor

$$\mathbf{E} = (\mathbf{C} - \mathbf{tensor.eye}) ./ 2$$

Almansi strain tensor

$$\mathbf{e} = (\mathbf{tensor.eye} - \mathbf{inv}(\mathbf{B})) ./ 2$$

engineering strain

$$\mathbf{eps} = \mathbf{F.sym} - \mathbf{tensor.eye}$$

true, logarithmic or Hencky strain tensor

$$\mathbf{H} = \mathbf{logm}(\mathbf{U})$$

```
H = tensor (xyz)
      rank: 2 (3 x 3)
```

```
*10^-3
```

```
-0.686  26.192      0
```

```
26.192   0.686      0
```

```
0         0         0
```

Strain Tensors

right Cauchy-Green stretch tensor $\mathbf{C} = \mathbf{U}^2 = \mathbf{F}'\mathbf{F}$

$$\mathbf{C} = \mathbf{F}' \cdot * \mathbf{F}$$

left Cauchy-Green stretch tensor $\mathbf{B} = \mathbf{V}^2 = \mathbf{F}\mathbf{F}'$

$$\mathbf{B} = \mathbf{F} \cdot * \mathbf{F}'$$

Green-Lagrange strain tensor

$$\mathbf{E} = (\mathbf{C} - \mathbf{tensor.eye}) ./ 2$$

Almansi strain tensor

$$\mathbf{e} = (\mathbf{tensor.eye} - \mathbf{inv}(\mathbf{B})) ./ 2$$

engineering strain

$$\mathbf{eps} = \mathbf{F}.\mathbf{sym} - \mathbf{tensor.eye}$$

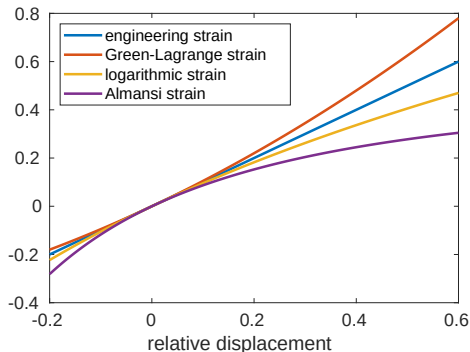
true, logarithmic or Hencky strain tensor

$$\mathbf{H} = \mathbf{logm}(\mathbf{U})$$

```
H = tensor (xyz)  
rank: 2 (3 x 3)
```

$*10^{-3}$

-0.686	26.192	0
26.192	0.686	0
0	0	0



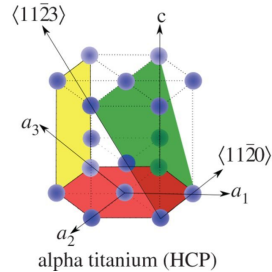
Slip Systems

A slip system is specified by a normal direction $\mathbf{n}$ and a slip direction $\mathbf{d}$ orthogonal to $\mathbf{n}$

$\mathbf{n} = \text{Miller}(0, 0, 0, 1, \text{cs}, 'hkl')$

$\mathbf{d} = \text{Miller}(1, 1, -2, 0, \text{cs}, 'uvw')$

$\text{sS} = \text{slipSystem}(\mathbf{d}, \mathbf{n})$



- {0001} basal slip plane
- {1010} prismatic slip plane
- {1011} pyramidal slip plane

```
sS = slipSystem (hcp)
```

U	V	T	W		H	K	I	L	CRSS
1	1	-2	0		0	0	0	1	1

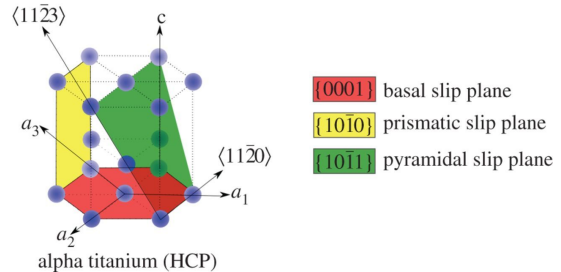
Slip Systems

A slip system is specified by a normal direction **n** and a slip direction **d** orthogonal to **n**

```
n = Miller(0,0,0,1,cs, 'hkl')  
d = Miller(1,1,-2,0,cs, 'uvw')  
sS = slipSystem(d,n)
```

build in fcc, bcc and hcp slip systems

```
sS = slipSystem.pyramidalA(cs)
```



```
sS = slipSystem (hcp)
```

U	V	T	W	H	K	I	L	CRSS
2	-1	-1	0	0	1	-1	1	1

Slip Systems

A slip system is specified by a normal direction **n** and a slip direction **d** orthogonal to **n**

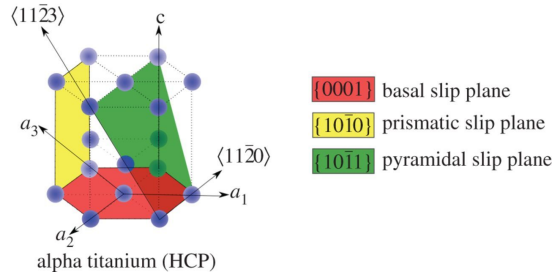
```
n = Miller(0,0,0,1,cs, 'hkl')  
d = Miller(1,1,-2,0,cs, 'uvw')  
sS = slipSystem(d,n)
```

build in fcc, bcc and hcp slip systems

```
sS = slipSystem.pyramidalA(cs)
```

symmetrically equivalent slip systems

```
sS.symmetrise
```



```
sS = slipSystem(hcp)
```

size: 12 x 1

U	V	T	W	H	K	I	L	CRSS
-2	1	1	0	0	1	-1	-1	1
2	-1	-1	0	0	1	-1	-1	1
-2	1	1	0	0	1	-1	1	1
2	-1	-1	0	0	1	-1	1	1
-1	-1	2	0	1	-1	0	1	1
1	1	-2	0	1	-1	0	1	1
-1	-1	2	0	-1	1	0	1	1
1	1	-2	0	-1	1	0	1	1
-1	2	-1	0	-1	0	1	1	1
1	-2	1	0	-1	0	1	1	1
-1	2	-1	0	1	0	-1	1	1
1	-2	1	0	1	0	-1	1	1

Slip Systems

A slip system is specified by a normal direction $\mathbf{n}$ and a slip direction $\mathbf{d}$ orthogonal to $\mathbf{n}$

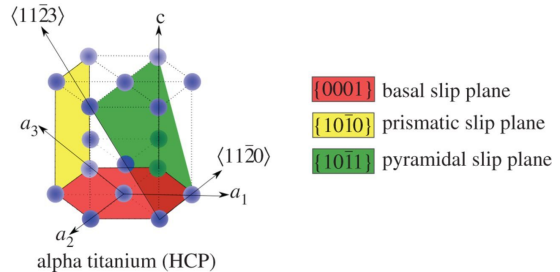
```
n = Miller(0,0,0,1,cs, 'hkl')  
d = Miller(1,1,-2,0,cs, 'uvw')  
sS = slipSystem(d,n)
```

build in fcc, bcc and hcp slip systems

```
sS = slipSystem.pyramidalA(cs)
```

symmetrically equivalent slip systems

```
sS.symmetrise('antipodal')
```



```
sS = slipSystem (hcp)  
size: 6 x 1
```

U	V	T	W	H	K	I	L	CRSS
-2	1	1	0	0	1	-1	-1	1
-2	1	1	0	0	1	-1	1	1
1	1	-2	0	1	-1	0	1	1
1	1	-2	0	-1	1	0	1	1
-1	2	-1	0	-1	0	1	1	1
-1	2	-1	0	1	0	-1	1	1

Slip Systems

A slip system is specified by a normal direction **n** and a slip direction **d** orthogonal to **n**

```
n = Miller(0,0,0,1,cs, 'hkl')  
d = Miller(1,1,-2,0,cs, 'uvw')  
sS = slipSystem(d,n)
```

build in fcc, bcc and hcp slip systems

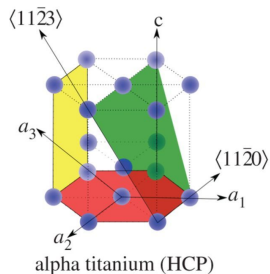
```
sS = slipSystem.pyramidalA(cs)
```

symmetrically equivalent slip systems

```
sS.symmetrise('antipodal')
```

transform to specimen reference frame

```
orientation.rand(cs) * sS
```



- {0001}** basal slip plane
- {101̄0}** prismatic slip plane
- {101̄1}** pyramidal slip plane

```
ans = slipSystem(xyz)
```

x	y	z		x	y	z
-0.03	-1.03	-0.97		-1.38	0.74	-0.74

Slip Systems

A slip system is specified by a normal direction **n** and a slip direction **d** orthogonal to **n**

```
n = Miller(0,0,0,1,cs, 'hkl')  
d = Miller(1,1,-2,0,cs, 'uvw')  
sS = slipSystem(d,n)
```

build in fcc, bcc and hcp slip systems

```
sS = slipSystem.pyramidalA(cs)
```

symmetrically equivalent slip systems

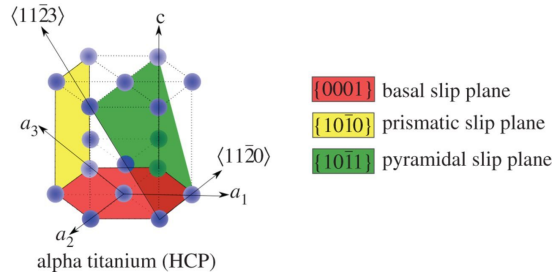
```
sS.symmetrise('antipodal')
```

transform to specimen reference frame

```
orientation.rand(cs) * sS
```

Schmid or deformation tensor

```
m = SchmidTensor(sS)
```



```
ans = velocityGradientTensor (hcp)  
rank : 2 (3 x 3)  
  
*10^-2  
32.73  56.69  56.69  
-18.9  -32.73 -32.73  
0       0       0
```

Schmid Factor

The Schmid factor is the resolved shear stress τ for a specific tension direction $\mathbf{r}$

$$\tau = \cos \angle(\mathbf{r}, \mathbf{n}) \cdot \cos \angle(\mathbf{r}, \mathbf{d}) = (\mathbf{r}^T \mathbf{n}) \cdot (\mathbf{r}^T \mathbf{d})$$

```
r = vector3d.Z  
tau = dot(n, r) .* dot(d, r)
```

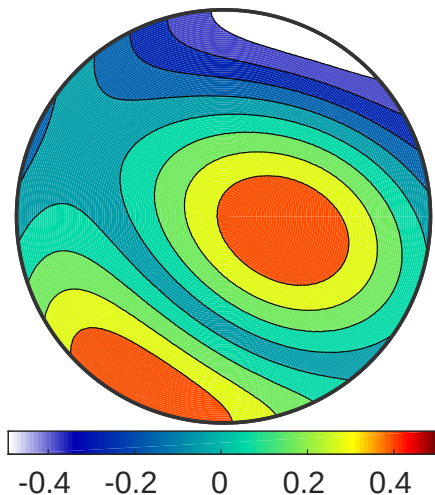
Schmid Factor

The Schmid factor is the resolved shear stress τ for a specific tension direction $\mathbf{r}$

$$\tau = \cos \angle(\mathbf{r}, \mathbf{n}) \cdot \cos \angle(\mathbf{r}, \mathbf{d}) = (\mathbf{r}^T \mathbf{n}) \cdot (\mathbf{r}^T \mathbf{d})$$

```
r = vector3d.Z  
tau = dot(n, r) .* dot(d, r)
```

```
tau = sS.SchmidFactor(r)  
plot(sS.SchmidFactor)
```



Schmid Factor

The Schmid factor is the resolved shear stress τ for a specific tension direction $\mathbf{r}$

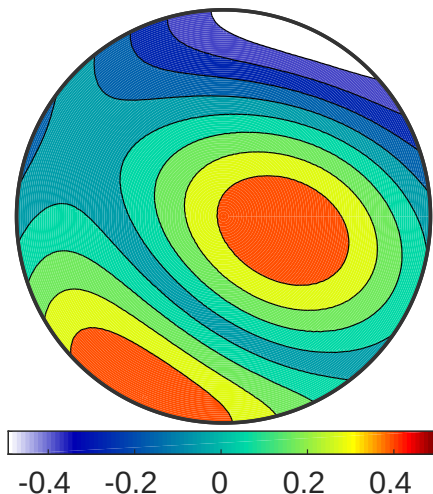
$$\tau = \cos \angle(\mathbf{r}, \mathbf{n}) \cdot \cos \angle(\mathbf{r}, \mathbf{d}) = (\mathbf{r}^T \mathbf{n}) \cdot (\mathbf{r}^T \mathbf{d})$$

```
r = vector3d.Z  
tau = dot(n, r) .* dot(d, r)
```

```
tau = sS.SchmidFactor(r)  
plot(sS.SchmidFactor)
```

For general stress σ we have $\tau = m : \sigma$

```
sigma = stressTensor.uniaxial(r)  
tau = EinsteinSum(m, [-1 -2], ...  
                  sigma, [-1, -2])  
tau = m : sigma  
tau = sS.SchmidFactor(sigma)
```



Schmid Factor

The Schmid factor is the resolved shear stress τ for a specific tension direction $\mathbf{r}$

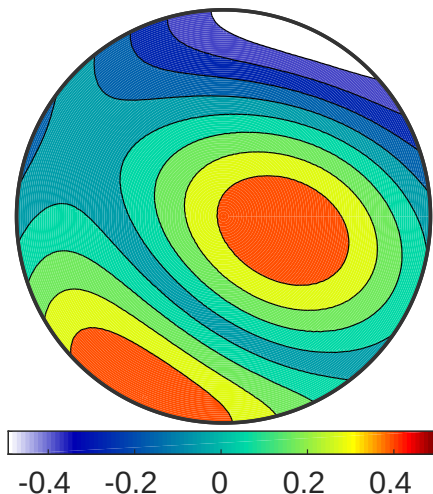
$$\tau = \cos \angle(\mathbf{r}, \mathbf{n}) \cdot \cos \angle(\mathbf{r}, \mathbf{d}) = (\mathbf{r}^T \mathbf{n}) \cdot (\mathbf{r}^T \mathbf{d})$$

```
r = vector3d.Z  
tau = dot(n, r) .* dot(d, r)
```

```
tau = sS.SchmidFactor(r)  
plot(sS.SchmidFactor)
```

For general stress σ we have $\tau = m : \sigma$

```
sigma = stressTensor.uniaxial(r)  
tau = (ori * m) : sigma  
tau = m : inv(ori) * sigma
```



Schmid Factor

The Schmid factor is the resolved shear stress τ for a specific tension direction $\mathbf{r}$

$$\tau = \cos \angle(\mathbf{r}, \mathbf{n}) \cdot \cos \angle(\mathbf{r}, \mathbf{d}) = (\mathbf{r}^T \mathbf{n}) \cdot (\mathbf{r}^T \mathbf{d})$$

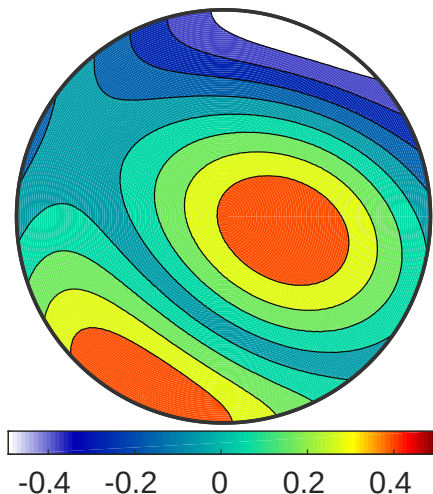
```
r = vector3d.Z  
tau = dot(n, r) .* dot(d, r)
```

```
tau = sS.SchmidFactor(r)  
plot(sS.SchmidFactor)
```

For general stress σ we have $\tau = m : \sigma$

```
sigma = stressTensor.uniaxial(r)  
tau = (ori * m) : sigma  
tau = m : inv(ori) * sigma
```

```
tau = SchmidFactor(ori*sS, sigma)
```



Maximum Schmid Factor

the Schmid factor for a list of tension directions results in a $\text{length}(r) \times \text{length}(sS)$ matrix tau

```
sS = sS.symmetrise( 'antipodal' )  
r = plotS2Grid( 'upper' )  
tau = sS.SchmidFactor( r )
```

maximum Schmidfactor over the second dimension

```
[tau_max, id] = max(abs( tau ), [], 2)  
contourf( r, tau_max )
```

the active slip system

```
contourf( r, id, 'contours', 12 )  
hold on  
quiver( r, sS( id ).n, 'color', 'r' );  
quiver( r, sS( id ).b, 'color', 'g' );  
hold off
```

Maximum Schmid Factor

the Schmid factor for a list of tension directions results in a $\text{length}(r) \times \text{length}(sS)$ matrix `tau`

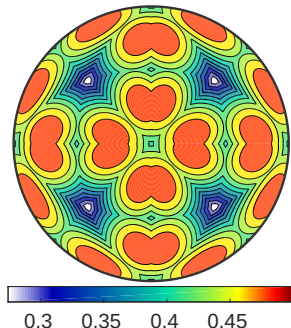
```
sS = sS.symmetrise('antipodal')  
r = plotS2Grid('upper')  
tau = sS.SchmidFactor(r)
```

maximum Schmidfactor over the second dimension

```
[tau_max, id] = max(abs(tau), [], 2)  
contourf(r, tau_max)
```

the active slip system

```
contourf(r, id, 'contours', 12)  
hold on  
quiver(r, sS(id).n, 'color', 'r');  
quiver(r, sS(id).b, 'color', 'g');  
hold off
```



Maximum Schmid Factor

the Schmid factor for a list of tension directions results in a $\text{length}(r) \times \text{length}(sS)$ matrix `tau`

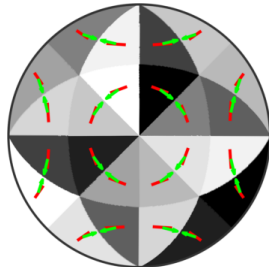
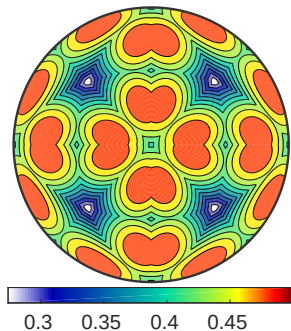
```
sS = sS.symmetrise('antipodal')  
r = plotS2Grid('upper')  
tau = sS.SchmidFactor(r)
```

maximum Schmidfactor over the second dimension

```
[tau_max, id] = max(abs(tau), [], 2)  
contourf(r, tau_max)
```

the active slip system

```
contourf(r, id, 'contours', 12)  
hold on  
quiver(r, sS(id).n, 'color', 'r');  
quiver(r, sS(id).b, 'color', 'g');  
hold off
```



Stress Based Analysis - Route 1

```
sigma = stressTensor.uniaxial(vector3d.X)
sS = symmetrise(slipSystem.fcc(ebsd.CS))
```

```
sS = slipSystem (fcc)
```

```
size: 24 x 1
```

u	v	w	h	k	l
0	1	-1	1	1	1
-1	0	1	1	1	1
1	-1	0	1	1	1
0	-1	1	1	1	1
1	0	-1	1	1	1
-1	1	0	1	1	1
1	-1	0	1	1	-1
1	0	1	1	1	-1
0	1	1	1	1	-1
-1	0	-1	1	1	-1
0	-1	-1	1	1	-1
-1	1	0	1	1	-1
0	1	-1	-1	1	1
1	0	1	-1	1	1
1	1	0	-1	1	1
-1	0	-1	-1	1	1
-1	-1	0	-1	1	1
0	-1	1	-1	1	1
-1	0	1	1	-1	1
1	1	0	1	-1	1
0	1	1	1	1	1

Stress Based Analysis - Route 1

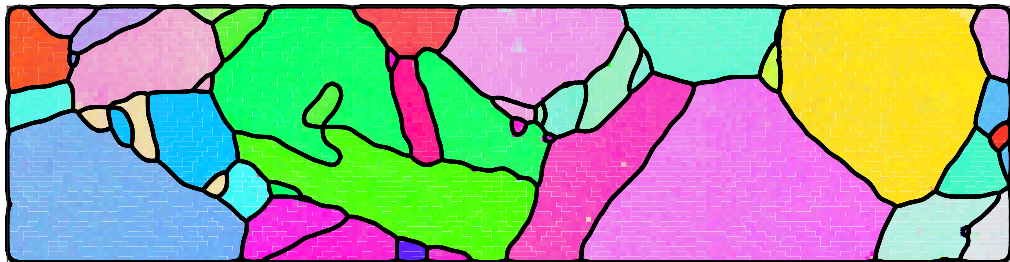
```
sigma = stressTensor.uniaxial(vector3d.X)
```

```
sS = symmetrise(slipSystem.fcc(ebsd.CS))
```

```
sSGrain = grains.meanOrientation .* sS
```

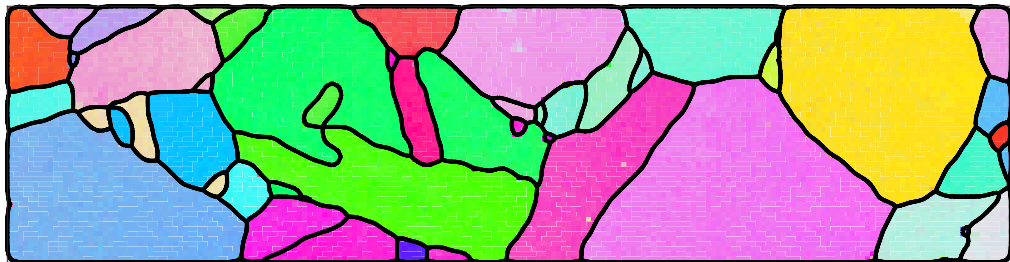
```
sSGrain = slipSystem (xyz)
```

```
size: 71 x 24
```



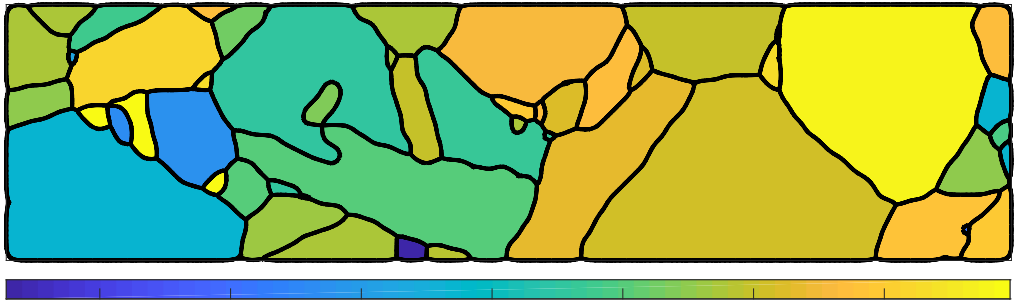
Stress Based Analysis - Route 1

```
sigma = stressTensor.uniaxial(vector3d.X)  
sS = symmetrise(slipSystem.fcc(ebsd.CS))  
sSGrain = grains.meanOrientation .* sS  
SF = sSGrain.SchmidFactor(sigma);
```



Stress Based Analysis - Route 1

```
sigma = stressTensor.uniaxial(vector3d.X)  
sS = symmetrise(slipSystem.fcc(ebsd.CS))  
sSGrain = grains.meanOrientation .* sS  
SF = sSGrain.SchmidFactor(sigma);  
[maxSF, active] = max(SF, [], 2);  
plot(grains, maxSF)
```



Stress Based Analysis - Route 1

```
sigma = stressTensor.uniaxial(vector3d.X)
```

```
sS = symmetrise(slipSystem.fcc(ebsd.CS))
```

```
sSGrain = grains.meanOrientation .* sS
```

```
SF = sSGrain.SchmidFactor(sigma);
```

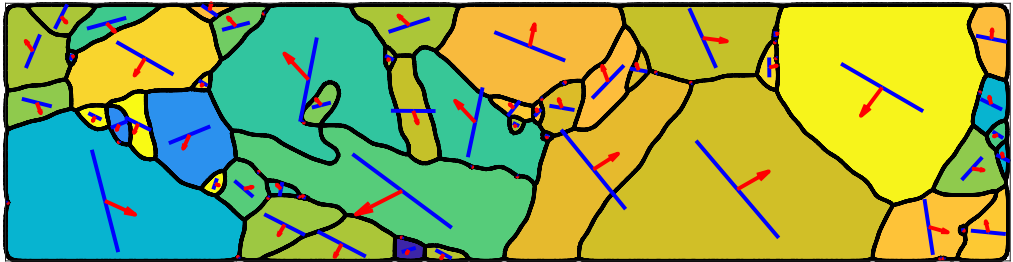
```
[maxSF, active] = max(SF, [], 2);
```

```
plot(grains, maxSF)
```

```
sSActive = grains.meanOrientation .* sS(active);
```

```
quiver(grains, sSActive.trace, 'color', 'blue')
```

```
quiver(grains, sSActive.b, 'color', 'red')
```

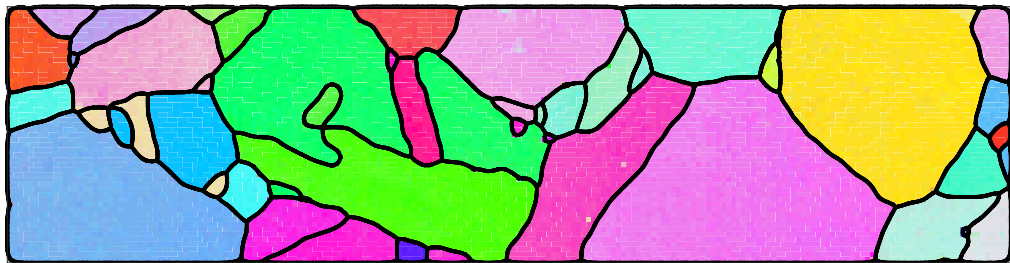


Stress Based Analysis - Route 2

```
sigma = stressTensor.uniaxial(vector3d.X)
```

```
sigma = stressTensor (xyz)  
rank: 2 (3 x 3)
```

```
1 0 0  
0 0 0  
0 0 0
```



Stress Based Analysis - Route 2

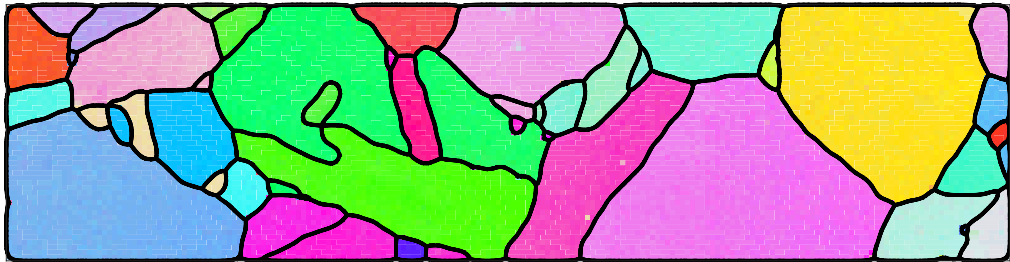
```
sigma = stressTensor.uniaxial(vector3d.X)
```

```
sigmaCrystal = inv(grains.meanOrientation) * sigma
```

```
sigmaCrystal = stressTensor (iron)
```

```
size      : 71 x 1
```

```
rank      : 2 (3 x 3)
```

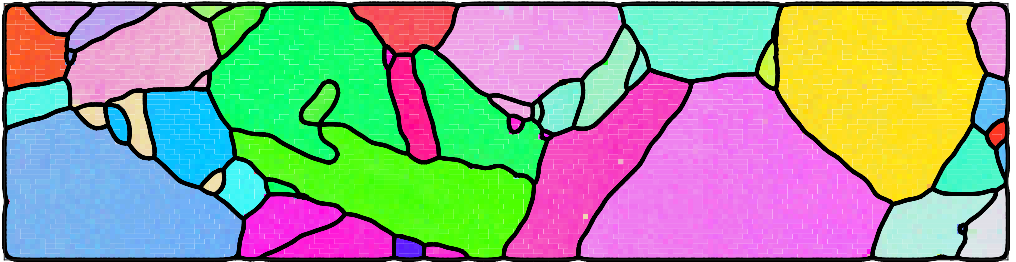


Stress Based Analysis - Route 2

```
sigma = stressTensor.uniaxial(vector3d.X)
```

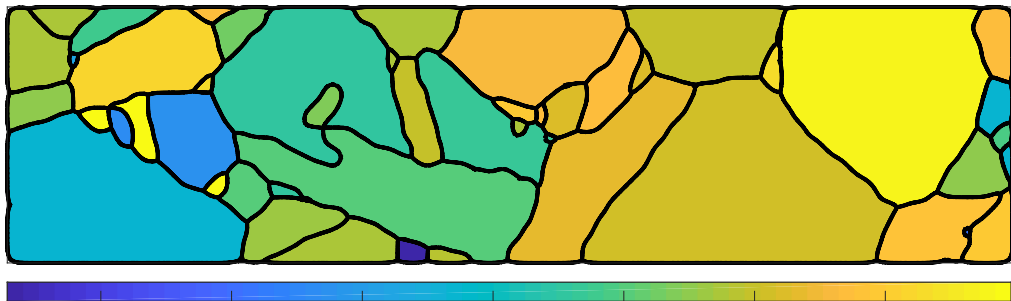
```
sigmaCrystal = inv(grains.meanOrientation) * sigma
```

```
SF = sS.SchmidFactor(sigmaCrystal);
```



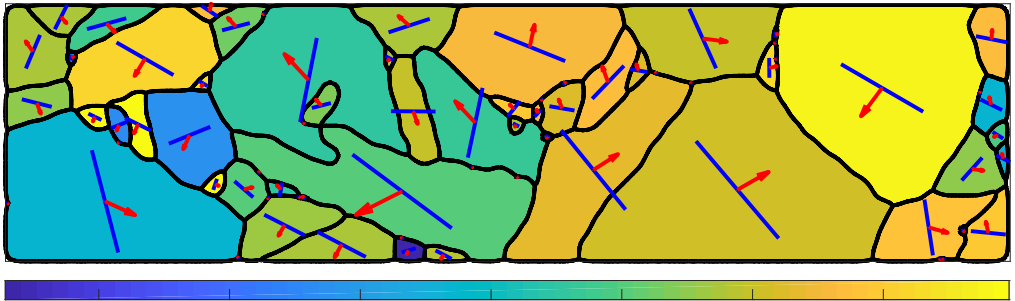
Stress Based Analysis - Route 2

```
sigma = stressTensor.uniaxial(vector3d.X)  
sigmaCrystal = inv(grains.meanOrientation) * sigma  
SF = sS.SchmidFactor(sigmaCrystal);  
[maxSF, active] = max(abs(SF), [], 2);  
plot(grains, maxSF)
```



Stress Based Analysis - Route 2

```
sigma = stressTensor.uniaxial(vector3d.X)
sigmaCrystal = inv(grains.meanOrientation) * sigma
SF = sS.SchmidFactor(sigmaCrystal);
[maxSF, active] = max(abs(SF), [], 2);
plot(grains, maxSF)
sSActive = grains.meanOrientation .* sS(active);
quiver(grains, sSActive.trace, 'color', 'blue')
quiver(grains, sSActive.b, 'color', 'red')
```

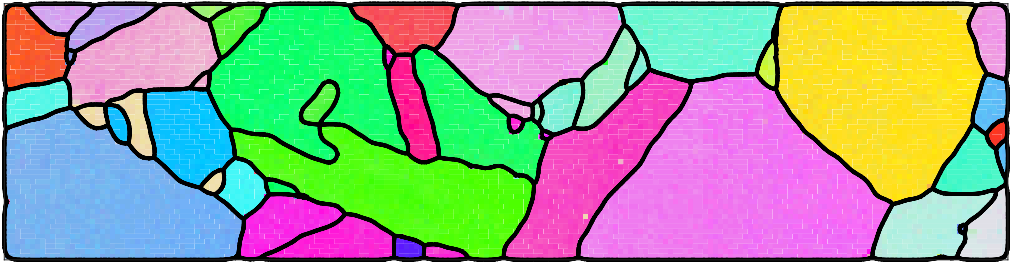


Strain Based Analysis

```
eps = strainTensor(diag([1,0,-1]))
```

```
sigma = strainTensor (xyz)  
rank: 2 (3 x 3)
```

```
1  0  0  
0  0  0  
0  0 -1
```



Strain Based Analysis

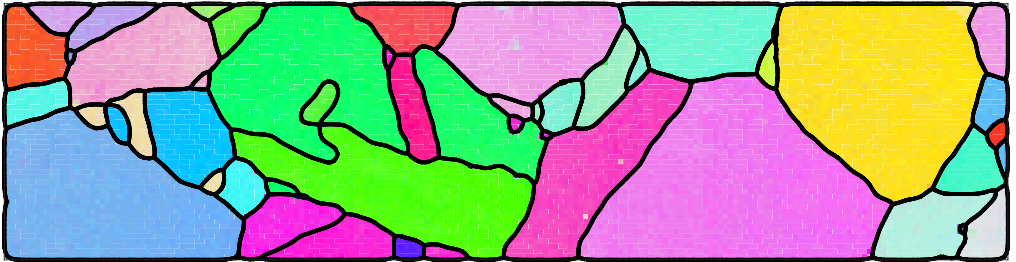
```
eps = strainTensor(diag([1,0,-1]))
```

```
epsCrystal = inv(grains.meanOrientation) * eps
```

```
epsCrystal = strainTensor (iron)
```

```
size      : 71 x 1
```

```
rank      : 2 (3 x 3)
```

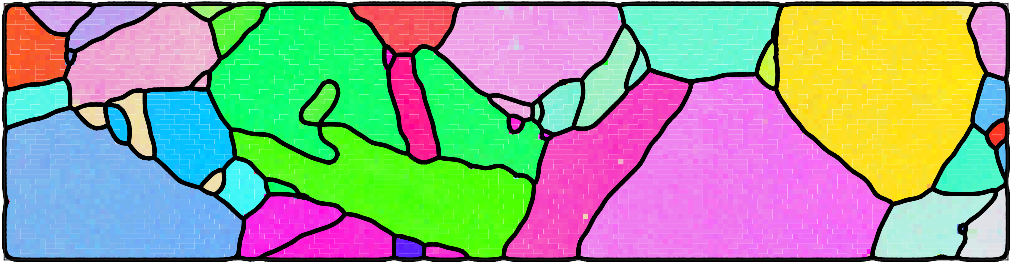


Strain Based Analysis

```
eps = strainTensor(diag([1,0,-1]))
```

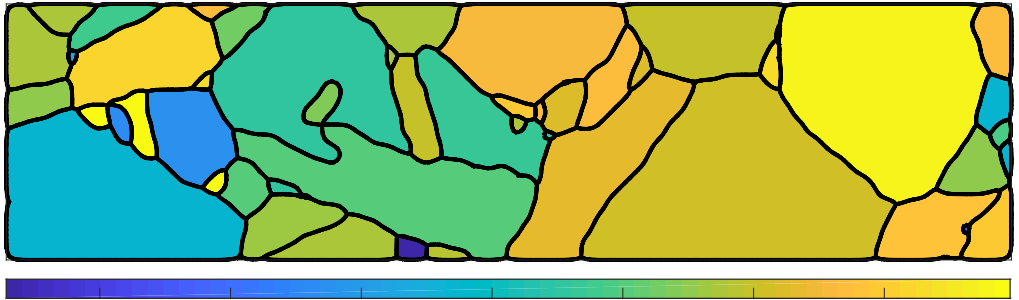
```
epsCrystal = inv(grains.meanOrientation) * eps
```

```
[TF, b, W] = calcTaylor(epsCrystal, sS);
```



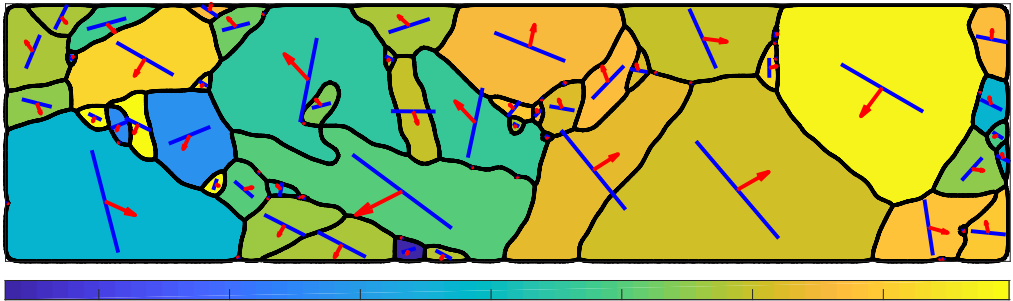
Strain Based Analysis

```
eps = strainTensor(diag([1,0,-1]))  
epsCrystal = inv(grains.meanOrientation) * eps  
[TF, b, W] = calcTaylor(epsCrystal, sS);  
plot(grains, TF)
```



Strain Based Analysis

```
eps = strainTensor(diag([1,0,-1]))  
epsCrystal = inv(grains.meanOrientation) * eps  
[TF, b, W] = calcTaylor(epsCrystal, sS);  
plot(grains, TF)  
[bMax, bMaxId] = max(b, [], 2);  
sSActive = grains.meanOrientation .* sS(bMaxId);  
quiver(grains, sSActive.b, 'color', 'red')  
quiver(grains, sSActive.trace, 'color', 'blue')
```



Rolling texture evolution

```
ori = orientation.rand(10000,CS)
h = Miller({0,0,1},{1,1,1}),CS)
plotPDF(ori, 'contourf')
```

the slip systems

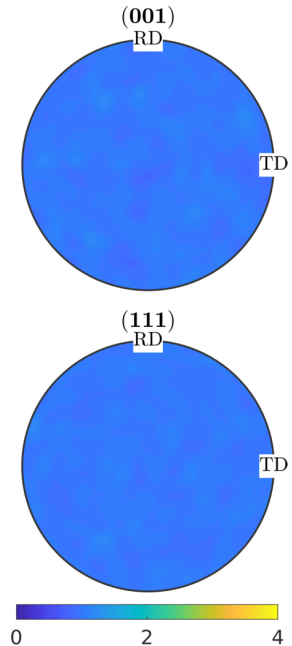
```
sS=symmetrise(slipSystem.fcc(CS))
```

30 percent strain

```
eps = strainTensor(diag([0.3 0 -0.3])) / 10
```

iterate deformation:

```
for i = 1:10
    [TF,b,W] = calcTaylor( inv(ori) * eps, sS);
    ori = ori .* orientation(-W);
    plotPDF(ori,h,'contourf')
end
```



Rolling texture evolution

```
ori = orientation.rand(10000,CS)
h = Miller({0,0,1},{1,1,1}),CS)
plotPDF(ori, 'contourf')
```

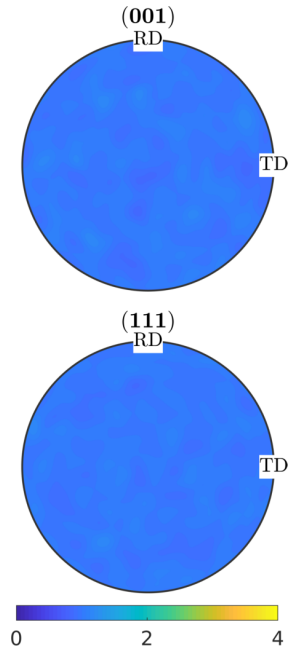
the slip systems

```
sS=symmetrise(slipSystem.fcc(CS))
```

```
sS = slipSystem (Austenite (fcc))
```

size: 12 x 1

u	v	w		h	k	l
0	1	-1		1	1	1
-1	0	1		1	1	1
1	-1	0		1	1	1
1	-1	0		1	1	-1
1	0	1		1	1	-1
0	1	1		1	1	-1
0	1	-1		-1	1	1
1	0	1		-1	1	1
1	1	0		-1	1	1
-1	0	1		1	-1	1
1	1	0		1	-1	1
0	1	1		1	-1	1



30 percent strain

Rolling texture evolution

```
ori = orientation.rand(10000,CS)
h = Miller({0,0,1},{1,1,1}),CS)
plotPDF(ori,'contourf')
```

the slip systems

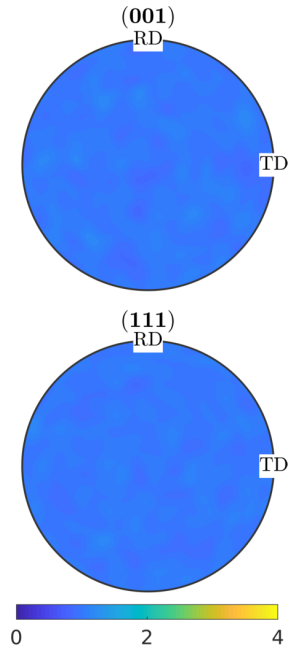
```
sS=symmetrise(slipSystem.fcc(CS))
```

30 percent strain

```
eps = strainTensor(diag([0.3 0 -0.3])) / 10
```

iterate deformation:

```
for i = 1:10
    [TF,b,W] = calcTaylor(inv(ori) * eps, sS);
    ori = ori .* orientation(-W);
    plotPDF(ori,h,'contourf')
end
```



Rolling texture evolution

```
ori = orientation.rand(10000,CS)
h = Miller({0,0,1},{1,1,1}),CS)
plotPDF(ori,'contourf')
```

the slip systems

```
sS=symmetrise(slipSystem.fcc(CS))
```

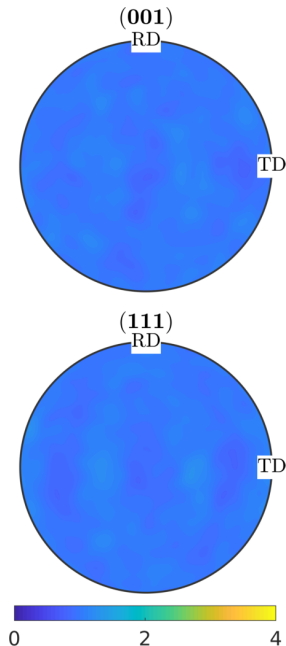
30 percent strain

```
eps = strainTensor(diag([0.3 0 -0.3])) / 10
```

iterate deformation:

```
for i = 1:10
    [TF,b,W] = calcTaylor(inv(ori) * eps, sS);
    ori = ori .* orientation(-W);
    plotPDF(ori,h,'contourf')
end
```

i=1



Rolling texture evolution

```
ori = orientation.rand(10000,CS)
h = Miller({0,0,1},{1,1,1}),CS)
plotPDF(ori, 'contourf')
```

the slip systems

```
sS=symmetrise(slipSystem.fcc(CS))
```

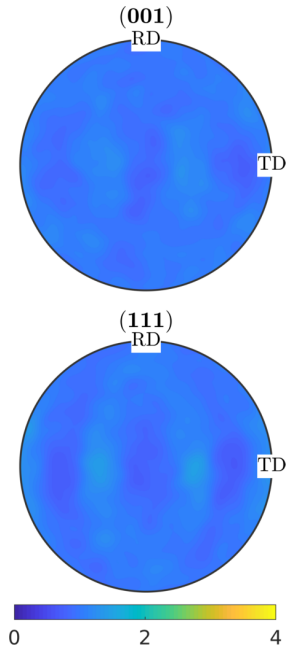
30 percent strain

```
eps = strainTensor(diag([0.3 0 -0.3])) / 10
```

iterate deformation:

```
for i = 1:10
    [TF,b,W] = calcTaylor(inv(ori) * eps, sS);
    ori = ori .* orientation(-W);
    plotPDF(ori,h,'contourf')
end
```

i=2



Rolling texture evolution

```
ori = orientation.rand(10000,CS)
h = Miller({0,0,1},{1,1,1}),CS)
plotPDF(ori,'contourf')
```

the slip systems

```
sS=symmetrise(slipSystem.fcc(CS))
```

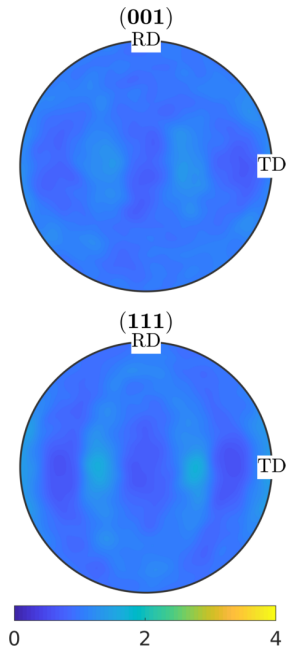
30 percent strain

```
eps = strainTensor(diag([0.3 0 -0.3])) / 10
```

iterate deformation:

```
for i = 1:10
    [TF,b,W] = calcTaylor(inv(ori) * eps, sS);
    ori = ori .* orientation(-W);
    plotPDF(ori,h,'contourf')
end
```

i=3



Rolling texture evolution

```
ori = orientation.rand(10000,CS)
h = Miller({0,0,1},{1,1,1}),CS)
plotPDF(ori,'contourf')
```

the slip systems

```
sS=symmetrise(slipSystem.fcc(CS))
```

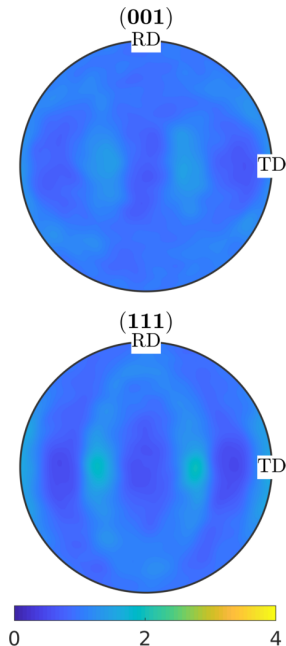
30 percent strain

```
eps = strainTensor(diag([0.3 0 -0.3])) / 10
```

iterate deformation:

```
for i = 1:10
    [TF,b,W] = calcTaylor(inv(ori) * eps, sS);
    ori = ori .* orientation(-W);
    plotPDF(ori,h,'contourf')
end
```

i=4



Rolling texture evolution

```
ori = orientation.rand(10000,CS)
h = Miller({0,0,1},{1,1,1}),CS)
plotPDF(ori,'contourf')
```

the slip systems

```
sS=symmetrise(slipSystem.fcc(CS))
```

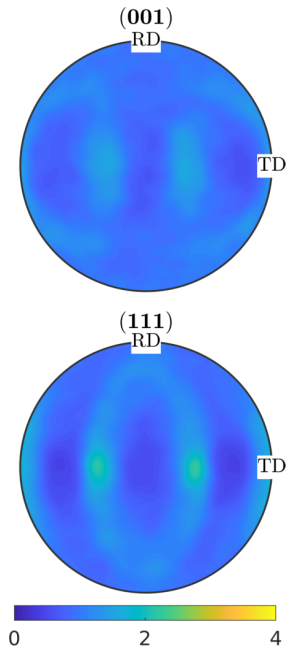
30 percent strain

```
eps = strainTensor(diag([0.3 0 -0.3])) / 10
```

iterate deformation:

```
for i = 1:10
    [TF,b,W] = calcTaylor(inv(ori) * eps, sS);
    ori = ori .* orientation(-W);
    plotPDF(ori,h,'contourf')
end
```

i=5



Rolling texture evolution

```
ori = orientation.rand(10000,CS)
h = Miller({0,0,1},{1,1,1}),CS)
plotPDF(ori,'contourf')
```

the slip systems

```
sS=symmetrise(slipSystem.fcc(CS))
```

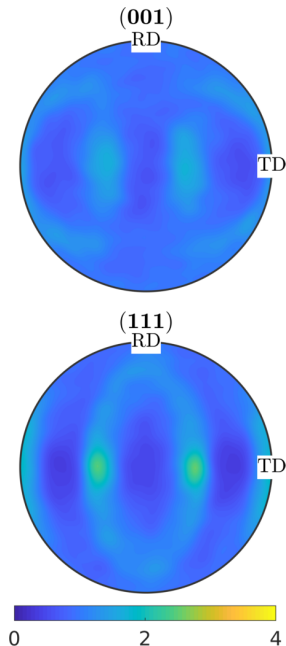
30 percent strain

```
eps = strainTensor(diag([0.3 0 -0.3])) / 10
```

iterate deformation:

```
for i = 1:10
    [TF,b,W] = calcTaylor(inv(ori) * eps, sS);
    ori = ori .* orientation(-W);
    plotPDF(ori,h,'contourf')
end
```

i=6



Rolling texture evolution

```
ori = orientation.rand(10000,CS)
h = Miller({0,0,1},{1,1,1}),CS)
plotPDF(ori,'contourf')
```

the slip systems

```
sS=symmetrise(slipSystem.fcc(CS))
```

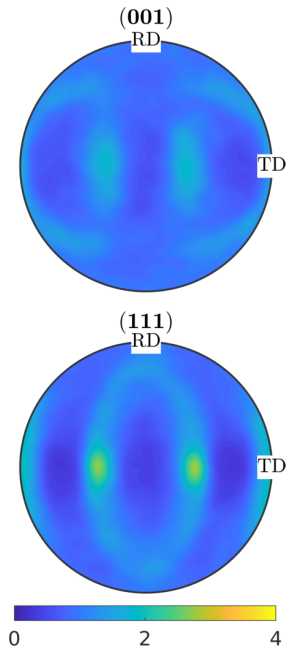
30 percent strain

```
eps = strainTensor(diag([0.3 0 -0.3])) / 10
```

iterate deformation:

```
for i = 1:10
    [TF,b,W] = calcTaylor(inv(ori) * eps, sS);
    ori = ori .* orientation(-W);
    plotPDF(ori,h,'contourf')
end
```

i=7



Rolling texture evolution

```
ori = orientation.rand(10000,CS)
h = Miller({0,0,1},{1,1,1}),CS)
plotPDF(ori,'contourf')
```

the slip systems

```
sS=symmetrise(slipSystem.fcc(CS))
```

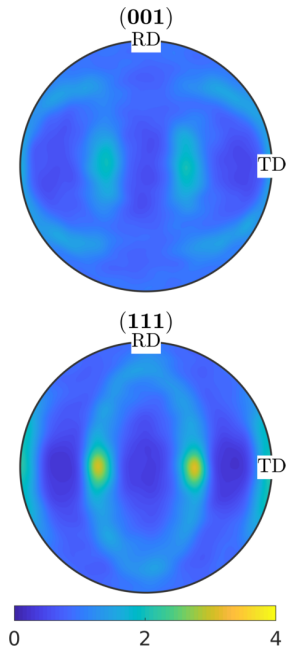
30 percent strain

```
eps = strainTensor(diag([0.3 0 -0.3])) / 10
```

iterate deformation:

```
for i = 1:10
    [TF,b,W] = calcTaylor(inv(ori) * eps, sS);
    ori = ori .* orientation(-W);
    plotPDF(ori,h,'contourf')
end
```

i=8



Rolling texture evolution

```
ori = orientation.rand(10000,CS)
h = Miller({0,0,1},{1,1,1}),CS)
plotPDF(ori,'contourf')
```

the slip systems

```
sS=symmetrise(slipSystem.fcc(CS))
```

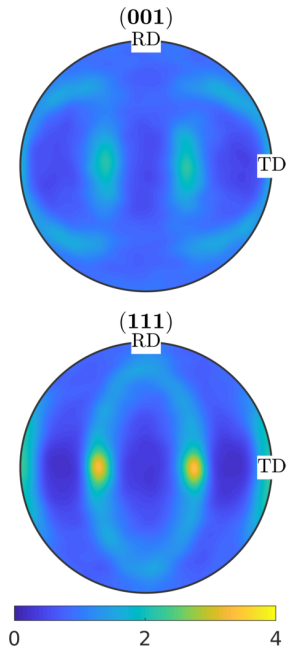
30 percent strain

```
eps = strainTensor(diag([0.3 0 -0.3])) / 10
```

iterate deformation:

```
for i = 1:10
    [TF,b,W] = calcTaylor(inv(ori) * eps, sS);
    ori = ori .* orientation(-W);
    plotPDF(ori,h,'contourf')
end
```

i=9



Rolling texture evolution

```
ori = orientation.rand(10000,CS)
h = Miller({0,0,1},{1,1,1}),CS)
plotPDF(ori,'contourf')
```

the slip systems

```
sS=symmetrise(slipSystem.fcc(CS))
```

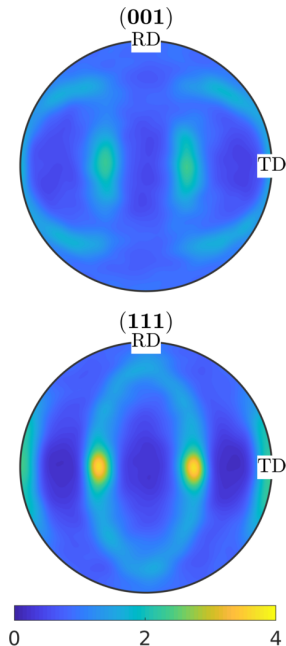
30 percent strain

```
eps = strainTensor(diag([0.3 0 -0.3])) / 10
```

iterate deformation:

```
for i = 1:10
    [TF,b,W] = calcTaylor(inv(ori) * eps, sS);
    ori = ori .* orientation(-W);
    plotPDF(ori,h,'contourf')
end
```

i=10



Texture Evolution - The Traditional Route

Ingredients

- ▶ family of slip systems (b_ℓ, n_ℓ)
- ▶ Schmid tensors $S_\ell = b_\ell \otimes d_\ell$
- ▶ strain tensor ε
- ▶ deformation law $\Omega(\text{ori})$
- ▶ initial ODF f_0

Algorithm

1. simulate orientations ori_k^0
2. update $\text{ori}_k^1 = \text{ori}_k^0 + \Omega(\text{ori}_k^0)$
3. estimate ODF f^1 from ori_k^1

```
cs = crystalSymmetry( '222' , [5 10 6] )  
sS = slipSystem( {1,0,0} , {0,0,1} , cs )  
S   = sS . SchmidTensor
```

```
S = velocityGradientTensor (olivine)  
rank: 2 (3 x 3)  
  
0 0 1  
0 0 0  
0 0 0
```

Texture Evolution - The Traditional Route

Ingredients

- ▶ family of slip systems (b_ℓ, n_ℓ)
- ▶ Schmid tensors $S_\ell = b_\ell \otimes d_\ell$
- ▶ strain tensor ε
- ▶ deformation law $\Omega(\text{ori})$
- ▶ initial ODF f_0

```
cs = crystalSymmetry('222',[5 10 6])  
sS = slipSystem({1,0,0},{0,0,1},cs)  
S  = sS.SchmidTensor
```

```
eps = strainTensor(diag(0.3,0,-0.3))
```

Algorithm

1. simulate orientations ori_k^0
2. update $\text{ori}_k^1 = \text{ori}_k^0 + \Omega(\text{ori}_k^0)$
3. estimate ODF f^1 from ori_k^1

Texture Evolution - The Traditional Route

Ingredients

- ▶ family of slip systems (b_ℓ, n_ℓ)
- ▶ Schmid tensors $S_\ell = b_\ell \otimes d_\ell$
- ▶ strain tensor ε
- ▶ deformation law $\Omega(\text{ori})$
- ▶ initial ODF f_0

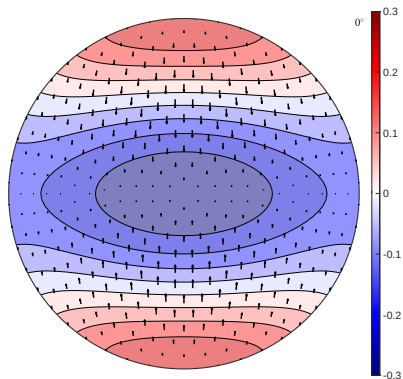
```
cs = crystalSymmetry( '222' , [5 10 6] )  
sS = slipSystem( {1,0,0} , {0,0,1} , cs )  
S   = sS . SchmidTensor
```

```
eps = strainTensor( diag(0.3,0,-0.3))
```

$$\Omega(\text{ori}) = (S_{\text{even}}(\text{ori}) : \varepsilon) \cdot S_{\text{odd}}$$

Algorithm

1. simulate orientations ori_k^0
2. update $\text{ori}_k^1 = \text{ori}_k^0 + \Omega(\text{ori}_k^0)$
3. estimate ODF f^1 from ori_k^1



Texture Evolution - The Traditional Route

Ingredients

- ▶ family of slip systems (b_ℓ, n_ℓ)
- ▶ Schmid tensors $S_\ell = b_\ell \otimes d_\ell$
- ▶ strain tensor ε
- ▶ deformation law $\Omega(\text{ori})$
- ▶ initial ODF f_0

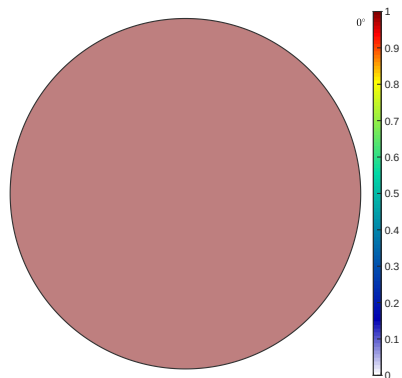
```
cs = crystalSymmetry('222',[5 10 6])  
sS = slipSystem({1,0,0},{0,0,1},cs)  
S  = sS.SchmidTensor
```

```
eps = strainTensor(diag(0.3,0,-0.3))
```

$$\Omega(\text{ori}) = (S_{\text{even}}(\text{ori}) : \varepsilon) \cdot S_{\text{odd}}$$

Algorithm

1. simulate orientations ori_k^0
2. update $\text{ori}_k^1 = \text{ori}_k^0 + \Omega(\text{ori}_k^0)$
3. estimate ODF f^1 from ori_k^1



Texture Evolution - The Traditional Route

Ingredients

- ▶ family of slip systems (b_ℓ, n_ℓ)
- ▶ Schmid tensors $S_\ell = b_\ell \otimes d_\ell$
- ▶ strain tensor ε
- ▶ deformation law $\Omega(\text{ori})$
- ▶ initial ODF f_0

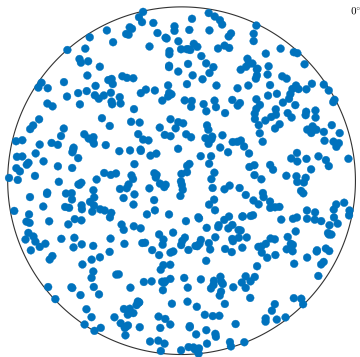
```
cs = crystalSymmetry('222',[5 10 6])  
sS = slipSystem({1,0,0},{0,0,1},cs)  
S = sS.SchmidTensor
```

```
eps = strainTensor(diag(0.3,0,-0.3))
```

$$\Omega(\text{ori}) = (S_{\text{even}}(\text{ori}) : \varepsilon) \cdot S_{\text{odd}}$$

Algorithm

1. simulate orientations ori_k^0
2. update $\text{ori}_k^1 = \text{ori}_k^0 + \Omega(\text{ori}_k^0)$
3. estimate ODF f^1 from ori_k^1



Texture Evolution - The Traditional Route

Ingredients

- ▶ family of slip systems (b_ℓ, n_ℓ)
- ▶ Schmid tensors $S_\ell = b_\ell \otimes d_\ell$
- ▶ strain tensor ε
- ▶ deformation law $\Omega(\text{ori})$
- ▶ initial ODF f_0

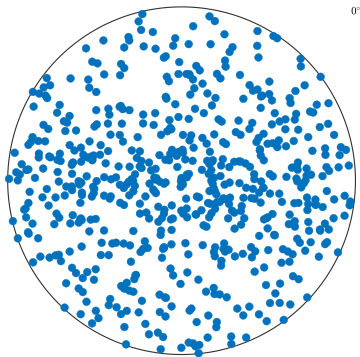
```
cs = crystalSymmetry( '222' , [5 10 6] )  
sS = slipSystem( {1,0,0} , {0,0,1} , cs )  
S   = sS . SchmidTensor
```

```
eps = strainTensor( diag(0.3,0,-0.3))
```

$$\Omega(\text{ori}) = (S_{\text{even}}(\text{ori}) : \varepsilon) \cdot S_{\text{odd}}$$

Algorithm

1. simulate orientations ori_k^0
2. update $\text{ori}_k^1 = \text{ori}_k^0 + \Omega(\text{ori}_k^0)$
3. estimate ODF f^1 from ori_k^1



Texture Evolution - The Traditional Route

Ingredients

- ▶ family of slip systems (b_ℓ, n_ℓ)
- ▶ Schmid tensors $S_\ell = b_\ell \otimes d_\ell$
- ▶ strain tensor ε
- ▶ deformation law $\Omega(\text{ori})$
- ▶ initial ODF f_0

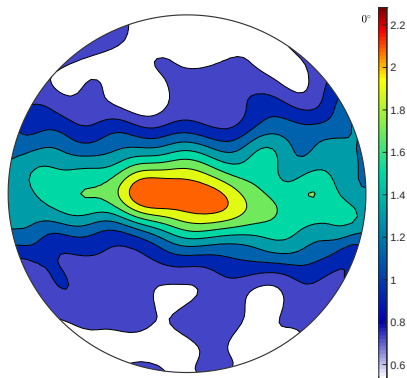
```
cs = crystalSymmetry( '222' , [5 10 6] )  
sS = slipSystem( {1,0,0} , {0,0,1} , cs )  
S   = sS . SchmidTensor
```

```
eps = strainTensor( diag(0.3,0,-0.3))
```

$$\Omega(\text{ori}) = (S_{\text{even}}(\text{ori}) : \varepsilon) \cdot S_{\text{odd}}$$

Algorithm

1. simulate orientations ori_k^0
2. update $\text{ori}_k^1 = \text{ori}_k^0 + \Omega(\text{ori}_k^0)$
3. estimate ODF f^1 from ori_k^1



Texture Evolution - The Traditional Route

Ingredients

- ▶ family of slip systems (b_ℓ, n_ℓ)
- ▶ Schmid tensors $S_\ell = b_\ell \otimes d_\ell$
- ▶ strain tensor ε
- ▶ deformation law $\Omega(\text{ori})$
- ▶ initial ODF f_0

```
cs = crystalSymmetry( '222' , [5 10 6] )  
sS = slipSystem( {1,0,0} , {0,0,1} , cs )  
S   = sS . SchmidTensor
```

```
eps = strainTensor( diag(0.3,0,-0.3))
```

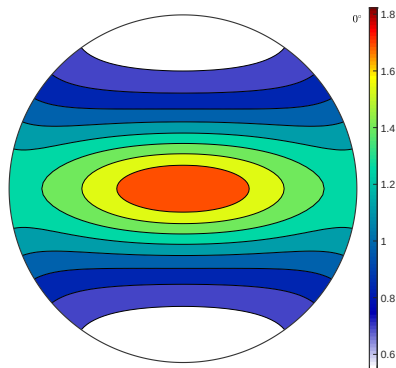
$$\Omega(\text{ori}) = (S_{\text{even}}(\text{ori}) : \varepsilon) \cdot S_{\text{odd}}$$

Algorithm

1. simulate orientations ori_k^0
2. update $\text{ori}_k^1 = \text{ori}_k^0 + \Omega(\text{ori}_k^0)$
3. estimate ODF f^1 from ori_k^1

Problems

- ▶ many orientations ori_k^0 required
- ▶ slow computation of $\Omega(\text{ori}_k^0)$
- ▶ ODF estimation not so easy

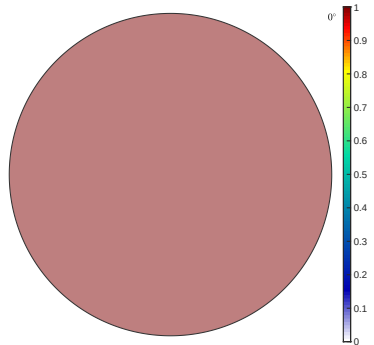
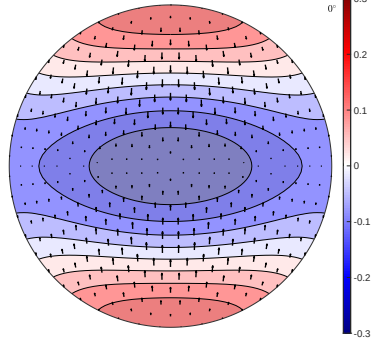


The Transport Equation

The continuity or transport equation

$$\frac{\partial}{\partial t} f = -\text{div}(f \cdot \Omega)$$

describes the change of the ODF $f(\text{ori}, t)$ at time t with respect to the spin tensor $\Omega(\text{ori})$



The Transport Equation

The continuity or transport equation

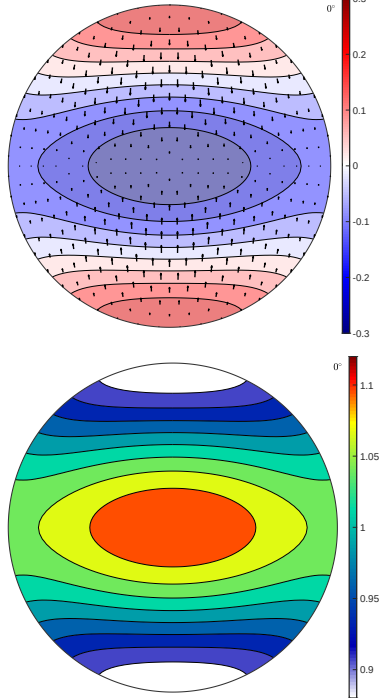
$$\frac{\partial}{\partial t} f = -\text{div}(f \cdot \Omega)$$

describes the change of the ODF $f(\text{ori}, t)$ at time t with respect to the spin tensor $\Omega(\text{ori})$

Euler method

- ▶ start with an initial odf f_0
- ▶ for $k = 1, \dots, N$ do

$$f_k = f_{k-1} - \frac{1}{N} \text{div}(f_k \cdot \Omega)$$



The Transport Equation

The continuity or transport equation

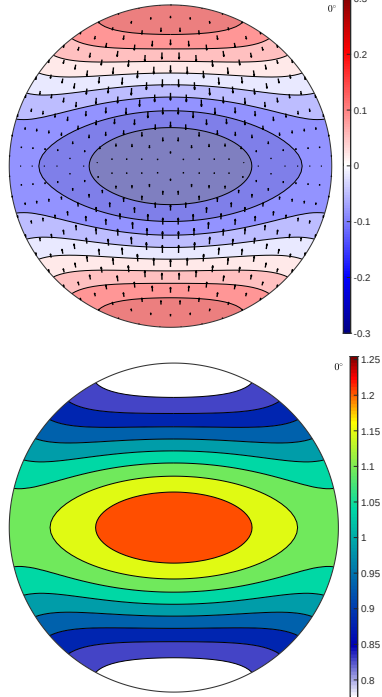
$$\frac{\partial}{\partial t} f = -\text{div}(f \cdot \Omega)$$

describes the change of the ODF $f(\text{ori}, t)$ at time t with respect to the spin tensor $\Omega(\text{ori})$

Euler method

- ▶ start with an initial odf f_0
- ▶ for $k = 1, \dots, N$ do

$$f_k = f_{k-1} - \frac{1}{N} \text{div}(f_k \cdot \Omega)$$



The Transport Equation

The continuity or transport equation

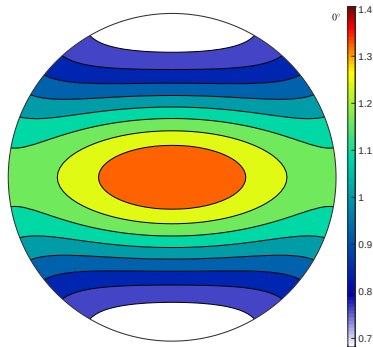
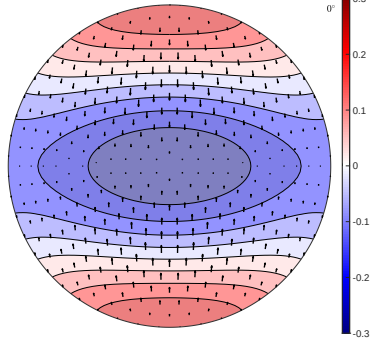
$$\frac{\partial}{\partial t} f = -\text{div}(f \cdot \Omega)$$

describes the change of the ODF $f(\text{ori}, t)$ at time t with respect to the spin tensor $\Omega(\text{ori})$

Euler method

- ▶ start with an initial odf f_0
- ▶ for $k = 1, \dots, N$ do

$$f_k = f_{k-1} - \frac{1}{N} \text{div}(f_k \cdot \Omega)$$



The Transport Equation

The continuity or transport equation

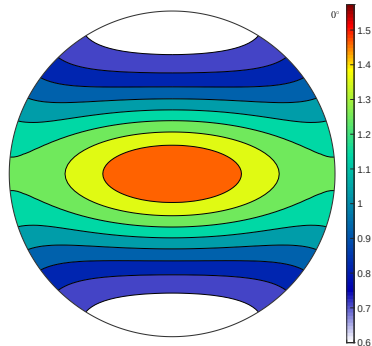
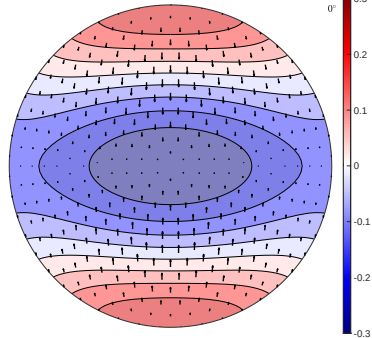
$$\frac{\partial}{\partial t} f = -\text{div}(f \cdot \Omega)$$

describes the change of the ODF $f(\text{ori}, t)$ at time t with respect to the spin tensor $\Omega(\text{ori})$

Euler method

- ▶ start with an initial odf f_0
- ▶ for $k = 1, \dots, N$ do

$$f_k = f_{k-1} - \frac{1}{N} \text{div}(f_k \cdot \Omega)$$



The Transport Equation

The continuity or transport equation

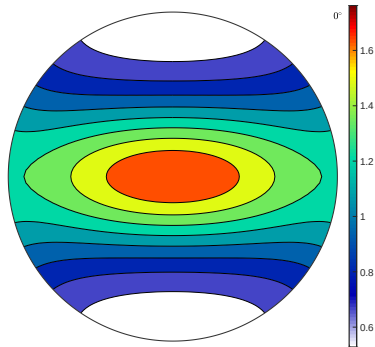
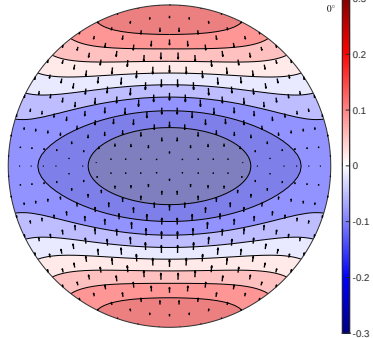
$$\frac{\partial}{\partial t} f = -\operatorname{div}(f \cdot \Omega)$$

describes the change of the ODF $f(\text{ori}, t)$ at time t with respect to the spin tensor $\Omega(\text{ori})$

Euler method

- ▶ start with an initial odf f_0
- ▶ for $k = 1, \dots, N$ do

$$f_k = f_{k-1} - \frac{1}{N} \operatorname{div}(f_k \cdot \Omega)$$



Geometrically Necessary Dislocations

```
ebsd = loadEBSD( 'DC06_2undef.ang' )
```

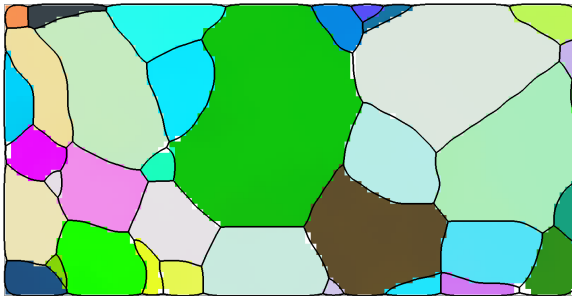
```
ebsd = EBSD
```

Phase	Orientations	Mineral	Symmetry
0	5123	Iron (Alpha)	432

Properties: ci, fit, iq, sem_signal, x, y, grainId

Scan unit : um

Grid size : 51 x 101



Geometrically Necessary Dislocations

```
ebsd = loadEBSD( 'DC06_2undef.ang' )
```

```
kappa = ebsd.curvature
```

```
kappa = curvatureTensor (xyz)
```

```
size: 51 x 101
```

```
unit: 1/um
```

```
rank: 2 (3 x 3)
```

```
kappa(2,3)
```

```
ans = curvatureTensor (xyz)
```

```
unit: 1/um
```

```
rank: 2 (3 x 3)
```

```
*10-5
```

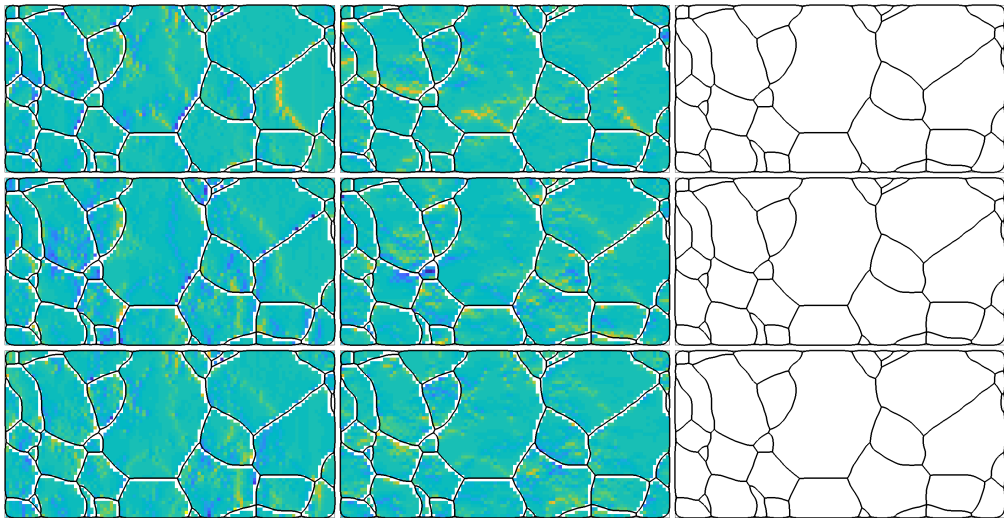
```
-7.262    12.678    NaN
```

```
-18.454   27.836   NaN
```

```
-20.467   25.478   NaN
```


Geometrically Necessary Dislocations

```
plot(ebsd , kappa {1,1})
```



Geometrically Necessary Dislocations

```
ebsd = loadEBSD( 'DC06_2undef.ang' )
```

```
kappa = ebsd.curvature
```

```
alpha = kappa.dislocationDensity
```

```
alpha = dislocationDensityTensor (xyz)  
size: 51 x 101  
unit: 1/um  
rank: 2 (3 x 3)
```

```
alpha(2,3)
```

```
ans = dislocationDensityTensor (xyz)  
unit: 1/um  
rank: 2 (3 x 3)  
  
*10^-5  
      NaN -18.454 -20.467  
12.678      NaN  25.478  
      NaN      NaN -20.574
```

Geometrically Necessary Dislocations

```
ebsd = loadEBSD( 'DC06_2undef.ang' )
```

```
kappa = ebsd.curvature
```

```
alpha = kappa.dislocationDensity
```

```
dS = dislocationSystem.bcc(ebsd.CS)
```

```
dS = dislocationSystem (Iron (alpha))
```

```
edge dislocations : 48 x 1
```

Burgers vector	line vector	energy
[1 -1 1]	[-2 -1 1]	2
[-1 1 1]	[0 1 -1]	2
[-1 1 1]	[-1 4 -5]	2

```
screw dislocations: 4 x 1
```

Burgers vector	energy
[-1 -1 -1]	1
[1 -1 1]	1
[-1 1 1]	1
[1 1 -1]	1

Geometrically Necessary Dislocations

```
ebsd = loadEBSD( 'DC06_2undef.ang' )
kappa = ebsd.curvature
alpha = kappa.dislocationDensity
dS = dislocationSystem.bcc(ebsd.CS)
dS(1).tensor

ans = dislocationDensityTensor (/*Iron (Alpha)*/)
    unit      : au
    rank      : 2 (3 x 3)

-1.1717  -0.5858   0.5858
 1.1717   0.5858  -0.5858
-1.1717  -0.5858   0.5858
```

Geometrically Necessary Dislocations

```
ebsd = loadEBSD( 'DC06_2undef.ang' )
```

```
kappa = ebsd.curvature
```

```
alpha = kappa.dislocationDensity
```

```
dS = dislocationSystem.bcc(ebsd.CS)
```

```
dSSample = ebsd.orientations * dS
```

```
dSSample = dislocationSystem (xyz)
```

```
edge dislocations : 5123 x 48
```

```
screw dislocations: 5123 x 4
```

Geometrically Necessary Dislocations

```
ebsd = loadEBSD( 'DC06_2undef.ang' )
```

```
kappa = ebsd.curvature
```

```
alpha = kappa.dislocationDensity
```

```
dS = dislocationSystem.bcc(ebsd.CS)
```

```
dSSample = ebsd.orientations * dS
```

```
[rho, factor] = kappa.fitDislocationSystems(dSSample);
```

Geometrically Necessary Dislocations

```
ebsd = loadEBSD( 'DC06_2undef.ang' )
```

```
kappa = ebsd.curvature
```

```
alpha = kappa.dislocationDensity
```

```
dS = dislocationSystem.bcc(ebsd.CS)
```

```
dSSample = ebsd.orientations * dS
```

```
[rho, factor] = kappa.fitDislocationSystems(dSSample);
```

```
alpha = sum(dSSample.tensor .* rho, 2)
```

```
alpha(2,3)
```

```
ans = dislocationDensityTensor (xyz)
```

```
unit: 1/um
```

```
rank: 2 (3 x 3)
```

```
*10-5
```

```
-28.855 -18.454 -20.467
```

```
12.678 6.243 25.478
```

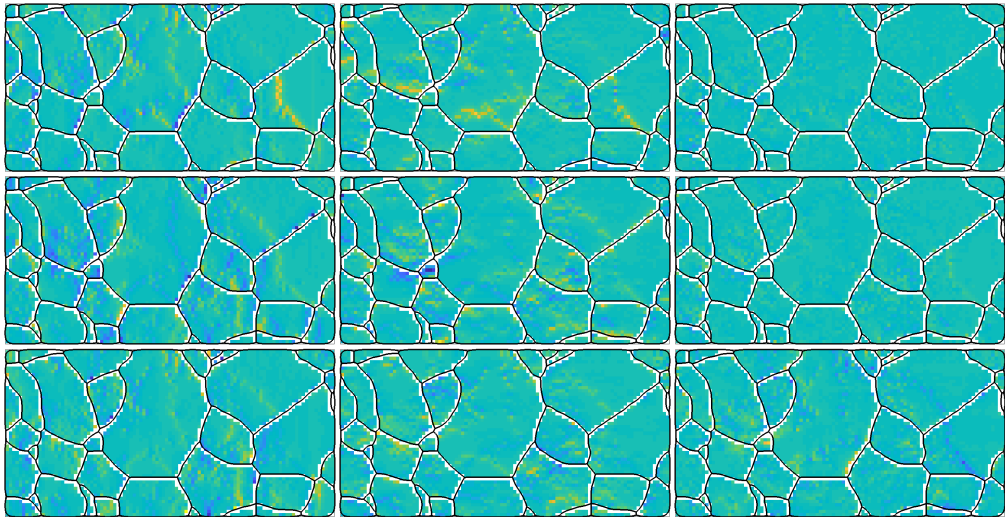
```
-1.337 -1.989 -20.574
```

Geometrically Necessary Dislocations

```
ebsd = loadEBSD( 'DC06_2undef.ang' )
kappa = ebsd.curvature
alpha = kappa.dislocationDensity
dS = dislocationSystem.bcc(ebsd.CS)
dSSample = ebsd.orientations * dS
[rho, factor] = kappa.fitDislocationSystems(dSSample);
alpha = sum(dSSample.tensor .* rho, 2)
kappa = alpha.curvature
kappa = curvatureTensor (xyz)
    size: 51 x 101
    unit: 1/um
    rank: 2 (3 x 3)
```


Geometrically Necessary Dislocations

```
plot(ebsd , kappa {1,1})
```



Geometrically Necessary Dislocations

```
plot(ebsd, factor * sum(abs(rho .* dSSample.u), 2))
```

